
Introduction to Backend, APIs, HTTP, and Python Environment (Theory)

Welcome to **FastAPI & MySQL**! In this session, we will establish the foundational concepts of web architecture, communication protocols, and your Python development environment.

1. Foundational Environment Setup (Python & VS Code)

To build backend services with FastAPI, you need a robust Python environment and code editor setup.

Follow these steps to set up your Python and FastAPI development environment on Windows:

Step 1: Install Python

1. Download the latest stable version of Python from the [Official Python Website](https://www.python.org/).
2. *Crucial:* During installation, check the box that says "**Add Python to PATH**".
3. Verify installation in your terminal:

```
python --version
```

Step 2: Install FastAPI and Uvicorn

With your environment, install FastAPI and `uvicorn` (the lightning-fast ASGI server used to run FastAPI):

```
pip install fastapi uvicorn pydantic
```

Step 3: Install Visual Studio Code (VS Code)

VS Code is the preferred code editor for modern web development.

1. Download it from the [Visual Studio Code Website](https://code.visualstudio.com/).
- *Essential Extensions:* Install the **Python** extension (by Microsoft) and **Pylance** for intelligent code completion, linting, and virtual environment management.

2. Core Backend & Web Concepts

A. What is a Backend?

In web development applications are split into two primary layers:

- **Frontend (Client-Side):** What the user sees and interacts with in their browser (built with React). It handles UI rendering, user input, and styling.
- **Backend (Server-Side):** The hidden engine running on a remote server or your local machine (built with FastAPI). It handles business logic, database communication, security, and machine learning model inference.
- **Database (Persistence):** Where data is permanently stored and retrieved (MySQL).

B. What is an API (Application Programming Interface)?

An API is a software intermediary that allows two applications to talk to each other. In our full-stack architecture:

- Your **React frontend** sends requests to the **FastAPI backend**.
- FastAPI processes the request, runs the machine learning model, queries MySQL, and returns structured data (usually in **JSON** format—JavaScript Object Notation) back to React.

C. What is HTTP (HyperText Transfer Protocol)?

HTTP is the foundational protocol used for data communication on the web. It operates on a **Request-Response Cycle**:

1. **Request:** The client (React) sends a message to the server containing:

- **URL / Endpoint:** (e.g., `http://localhost:8000/predict`)
- **HTTP Method:** Defines the action:
 - `GET` : Retrieve data from the server.
 - `POST` : Send data to the server (e.g., submitting feature inputs for prediction).
- **Payload / Headers:** The data body and metadata.

2. **Response:** The server (FastAPI) processes the request and sends back:

- **Status Code:** Indicates success or failure:
 - `200 OK` : Request succeeded.
 - `201 Created` : Resource successfully saved.
 - `400 Bad Request` : Invalid input payload.

- **500 Internal Server Error** : Server-side crash or exception.
- **Response Body**: The resulting JSON data.

D. Backend API Frameworks

These frameworks are built around Python's modern asynchronous capabilities (`async` / `await`). They are currently the go-to choices for highly concurrent workloads, real-time applications, and AI/LLM backends.

1. Modern & Async-First Frameworks

- **FastAPI**: Currently the most popular greenfield choice. It uses standard Python type hints to automatically validate data and generate interactive API documentation (Swagger UI/ReDoc). It is highly performant and widely adopted for modern API and AI workloads.

2. Enterprise & Full-Stack

When an API is deeply tied to a relational database, complex user authentication, and admin interfaces, full-stack frameworks are the standard.

- **Django REST Framework (DRF)**: The undisputed king of enterprise CRUD (Create, Read, Update, Delete) APIs. Built on top of Django, it provides powerful serializers, out-of-the-box authentication, pagination, and built-in ORM integration. It is highly structured and mature.

3. Lightweight Microframeworks

These frameworks provide the bare minimum needed to route HTTP requests and return responses, leaving architectural decisions entirely up to the developer.

- **Flask (with Flask-RESTful or Flask-RESTX)**: Flask is the classic Python microframework. While you can build APIs with plain Flask, extensions like Flask-RESTful add resource routing and payload validation. It remains a top choice for lean MVPs and highly customized tech stacks.

4. Specialized & Niche Frameworks

These frameworks serve specific architectural needs or historical use cases.

- **Tornado**: An asynchronous networking library and web framework built to maintain long-lived network connections (like WebSockets). It is highly specialized for real-time applications like chat platforms or live data feeds.
-

3. Introduction to FastAPI & Pydantic

FastAPI is a modern, high-performance Python web framework designed for building APIs.

- **Asynchronous Support (`async/def`):** Allows handling multiple concurrent requests efficiently without blocking server execution.
- **Automatic Interactive Documentation:** FastAPI automatically generates interactive API docs using Swagger UI (`/docs`) and ReDoc (`/redoc`) based on your Python type hints.
- **Pydantic Data Validation:** FastAPI uses Pydantic models to enforce strict data types on incoming request payloads, automatically catching invalid inputs (like negative house prices or missing text fields) before they reach your business logic or ML model.

Why Python for Backends?

Python is the undisputed language of data science, machine learning, and artificial intelligence. Using Python for your backend web framework means your API can seamlessly load, run, and serve serialized machine learning models (`.pkl` files) without needing to cross language barriers (like bridging Node.js to Python).

Why FastAPI?

Released in 2018, **FastAPI** has become a premier Python web framework due to several key advantages:

- **Blazing Speed:** Built on Starlette (for web handling) and Pydantic (for data validation), its performance is on par with NodeJS and Go.
- **Automatic Documentation:** FastAPI automatically generates interactive API documentation via Swagger UI (`/docs`) and ReDoc (`/redoc`) straight from your Python type hints.
- **Asynchronous Support (`async/await`):** Ideal for handling concurrent requests efficiently, especially when waiting for database queries or model inferences.

Why Uvicorn?

We use Uvicorn with FastAPI because of a strict division of labor in modern Python web development: **FastAPI is the application framework, but Uvicorn is the actual web server.**

1. The Networking Layer

FastAPI's job is purely to define routes, validate data using Pydantic, and execute your application's business logic. However, FastAPI does not know how to listen to a network port, accept raw incoming HTTP requests, or communicate over the internet.

Uvicorn acts as the networking layer. Think of it like a waiter in a restaurant: Uvicorn takes the order (the HTTP request) from the client, delivers it to the kitchen (FastAPI), waits for the food to be prepared, and then carries the finished meal (the HTTP response) back to the client.

2. The ASGI Bridge

For Uvicorn and FastAPI to communicate, they use a bridging standard called **ASGI** (Asynchronous Server Gateway Interface). Before ASGI, older Python frameworks like Django or Flask used WSGI, which processes requests synchronously (one at a time). Because FastAPI is designed around modern asynchronous Python (`async` / `await`), it requires an ASGI server to handle multiple connections, WebSockets, and long-running background tasks concurrently without blocking the main thread.

4. Core Concepts in Detail

A. Automatic Data Validation with Pydantic

FastAPI relies heavily on **Pydantic**, a data validation library that uses Python type annotations. When a client sends a JSON payload to your server, Pydantic automatically checks if the data matches your expected types (e.g., ensuring `feature1` is a float and not a string). If validation fails, FastAPI automatically returns a clear 422 Unprocessable Entity error.

```
from pydantic import BaseModel

class PredictionRequest(BaseModel):
    feature1: float
    feature2: float
```

B. Automatic OpenAPI Documentation

Because FastAPI inspects your Python type hints and Pydantic models, it automatically builds interactive documentation. Once your server runs, you can visit

`[http://127.0.0.1:8000/docs](http://127.0.0.1:8000/docs)` to test your endpoints directly in your browser without needing tools like Postman.

5. Reference Links & Further Reading

- **MDN Introduction to HTTP:** Comprehensive guide on web communication protocols.
- **Python Getting Started Guide:** Official documentation for Python installation and virtual environments.

- **FastAPI Official Documentation:** The definitive guide to getting started with FastAPI tutorials and advanced features.
 - **Pydantic Documentation:** Learn more about data parsing and strict type enforcement in Python.
 - **Python Virtual Environments Guide:** Official documentation on managing isolated Python environments.
-